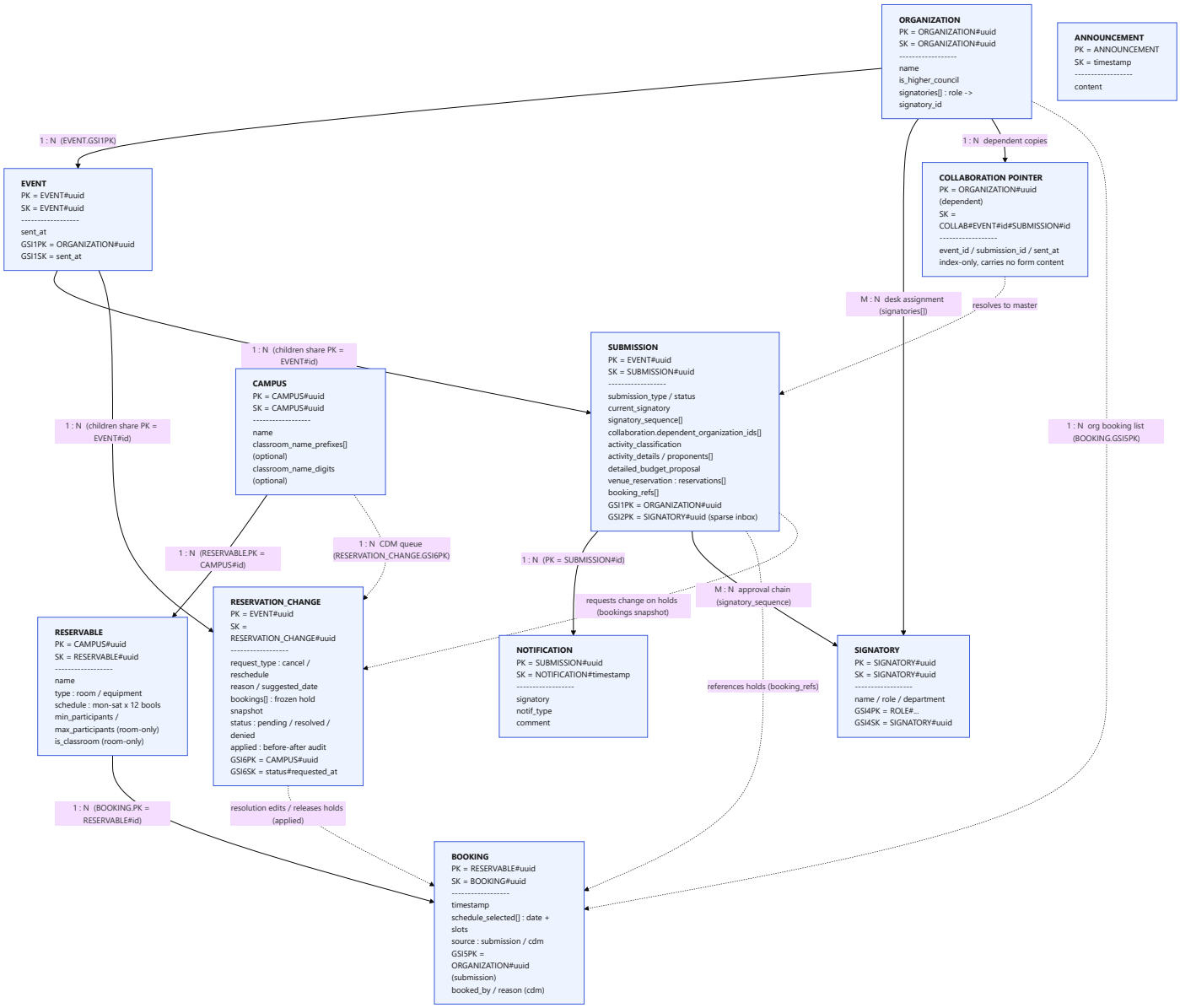
mapua-apex Data Model

DynamoDB single-table data model — conceptual ERD, object dictionary, and access patterns.

Generated 2026-10-14 01:30

Conceptual ERD



Data Objects & Properties

All objects live in a **single DynamoDB table**. Each item is identified by a partition key (`PK`) and a sort key (`SK`); several item types share one partition so related records are read together in a single query. There are **no foreign keys and no cascade**: related IDs are *copied* onto items and resolved at write time, so deleting one item never rewrites another.

ORGANIZATION

The student organization. Also the desk directory: it records which signatory person occupies each approval role for this org.

Property	Type	Notes
`PK`	string	`ORGANIZATION#uuid`
`SK`	string	`ORGANIZATION#uuid` (same as PK)
`name`	string	Display name
`is_higher_council`	boolean	Default `false` . When true, the Dean step is skipped in routing
`signatories`	list	Desk directory, one person per role: `{ role, signatory_id }`
`signatories[].role`	string	One of `adviser / dean / osaar / cdm / admin`
`signatories[].signatory_id`	string	Plain copy of a SIGNATORY id

Allowed desk roles are stored in the order `adviser → dean → osaar → cdm → admin` regardless of write order. `admin` may hold a desk but is **not** part of the approval sequence. `osaar` and `cdm` desks are campus-wide (from `OSAAR_SIGNATORY_ID` / `CDM_SIGNATORY_ID` in the API env), not stored per org.

EVENT

A filing/event that owns one or more submissions. Its partition also holds the event's SUBMISSION children.

Property	Type	Notes
`PK`	string	`EVENT#uuid`
`SK`	string	`EVENT#uuid` (same as PK)
`sent_at`	timestamp	Creation time (UTC)

Property	Type	Notes
`GSI1PK`	string	`ORGANIZATION#uuid` — owning org (also identifies the *proponent* org for collaborations)
`GSI1SK`	timestamp	`sent_at` — for newest-first org event lists

SUBMISSION

A proposal paper (e.g. SAAF) filed under an event. Carries the full form body plus routing state and index keys. This is the single source of truth for a submission, including any collaboration.

Identity, routing & indexes

Property	Type	Notes
`PK`	string	`EVENT#uuid` (parent event partition)
`SK`	string	`SUBMISSION#uuid`
`submission_type`	string	e.g. `saaf`
`sent_at`	timestamp	Submit time (UTC)
`status`	string	`pending / approved / denied / returned / finished / cancelled` — `finished` is set by the arcus post-evaluation finish endpoint; `cancelled` is terminal and set only by a CDM cancel-resolution of a RESERVATION_CHANGE (releases all holds, drops `booking_refs`, paper kept)
`current_signatory`	string	`SIGNATORY#uuid` holding the paper now
`signatory_sequence`	list	Ordered `SIGNATORY#uuid` routing snapshot, written at submit time
`collaboration.dependent_organization_ids`	list	`ORGANIZATION#uuid` dependents; empty/absent = no collaboration
`GSI1PK`	string	`ORGANIZATION#uuid` (proponent org)
`GSI1SK`	string	`SUBMISSION#uuid`
`GSI2PK`	string	`SIGNATORY#uuid` — sparse signatory inbox (present while `pending` / `returned`)

Property	Type	Notes
`GSI2SK`	timestamp	Desk queue time

activity_classification

Property	Type	Notes
`activity_type`	string	`co-curricular / extra-curricular`
`total_org_members`	number	
`nature`	string	`major / minor` — set by OSAAR via the classification PATCH, not by the student create body

proponents[] (list of people)

`id`, `position_title`, `first_name`, `middle_name`, `last_name`, `suffix`, `student_number`, `program_and_year`, `date_of_submission`, `department`, `position_of_applicant`, `org_or_course_section`, `contact_number`, `email_address`, `facebook_link`.

activity_details

`title_and_nature`, `description`, `objectives`, `venue`, `date_of_event`, `day_of_event`, `time_of_event`, `expected_participants`, `individual_contribution`, `proposed_budget`. `time_of_event` is 24-hour `"HH:MM - HH:MM"`; saved times fall between 7:00 AM and 9:00 PM, end not earlier than start, and start ≠ end.

institutional_alignment

`mission_statements { competitive, research, solutions }`, `core_values_explanation`, `peo_explanation`, `sdg_explanation`.

detailed_budget_proposal

`items[] { item_no, unit, quantity, price_per_unit, total }`, `grand_total`.

venue_reservation & booking_refs

Property	Type	Notes
`has_reservation`	boolean	Drives whether the CDM desk joins the routing chain
`reservations[]`	list	One entry per picked reservable: `{ reservable_id, campus_id, name, type, selections[], remarks }`
`reservations[].type`	string	`room` | `equipment`

Property	Type	Notes
`reservations[].selections[]`	list	`{ date: "YYYY-MM-DD", slots: [int 0-11] }` — 0-based indices into the 07:00–21:00 / 70-min day (12 slots)
`booking_refs[]`	list	Sparse reverse index `{ pk: "RESERVABLE#uuid", sk: "BOOKING#uuid" }`; present only while the paper holds BOOKINGs. Released + removed on deny/return/cancel-resolution, reconciled on re-edit and on a reschedule-resolution

Legacy items stored before this shape may still carry `equipment_requested` / `function_rooms` / `audiovisual_equipment`; reads tolerate both. On submit each selection is validated against the reservable's weekly template and existing BOOKINGs, then written as `source:"submission"` holds — a slot taken in the meantime aborts with **409** and persists nothing.

NOTIFICATION

One row in a submission's approval/timeline history. Stored in a partition keyed by the submission id (a separate partition from the event).

Property	Type	Notes
`PK`	string	`SUBMISSION#uuid`
`SK`	string	`NOTIFICATION#timestamp` (identity; UTC second precision)
`signatory`	string	`SIGNATORY#uuid` author (snapshot)
`notif_type`	string	`approved / fully approved / denied / returned / event scheduled / reservation change requested / reservation cancelled / reservation rescheduled / reservation change denied` — the four `reservation ...` types are system-only (written by RequestReservationChange / ResolveReservationChange, never client-postable)
`comment`	string	Mandatory when `denied` or `returned`

Approve / return / deny insert a NOTIFICATION row as a side effect. A PUT replaces `notif_type` / `comment` in place and does **not** change the SK.

SIGNATORY

A person (the signatory identity). Has **no** `organization_id`; desk assignment lives on `ORGANIZATION.signatories`.

Property	Type	Notes
`PK`	string	`SIGNATORY#uuid`
`SK`	string	`SIGNATORY#uuid` (same as PK)
`name`	string	
`role`	string	`adviser / cdm / dean / osaar / admin`
`department`	string	Deans only (e.g. `SOIT`)
`GSI4PK`	string	`ROLE#ADVISER / ROLE#CDM / ROLE#DEAN / ROLE#DEAN#{DEPARTMENT} / ROLE#OSAAR / ROLE#ADMIN`
`GSI4SK`	string	`SIGNATORY#uuid`

Other items copy the signatory id rather than join back to SIGNATORY: `ORGANIZATION.signatories[].signatory_id`, `SUBMISSION.current_signatory` / `GSI2PK`, and `NOTIFICATION.signatory`. Renaming keeps the same uuid; deleting a SIGNATORY does not rewrite those copies.

COLLABORATION POINTER

An index-only item, one per dependent organization, that makes the proponent's submission discoverable in a dependent's dashboard. It carries **no** form content — the master SUBMISSION stays the single source of truth.

Property	Type	Notes
`PK`	string	`ORGANIZATION#uuid` — the **dependent** org (not the proponent)
`SK`	string	`COLLAB#EVENT#uuid#SUBMISSION#uuid`
`event_id`	string	`EVENT#uuid`
`submission_id`	string	`SUBMISSION#uuid`
`sent_at`	timestamp	

Queried from the dependent's partition (`SK` begins `COLLAB#`); no GSI is needed because DynamoDB cannot index the N values of an array. A dependent may read the shared submission and its NOTIFICATION timeline but cannot edit or resubmit — only the proponent may.

ANNOUNCEMENT

A campus-wide announcement. All announcements share one partition; the SK (timestamp) is the identity. Admin Cognito group only.

Property	Type	Notes
`PK`	string	`ANNOUNCEMENT` (constant)
`SK`	string	`timestamp` — identity (`sent_at` in the API); newest-first list
`content`	string	

A PUT replaces `content` in place and does not change the SK.

CAMPUS

A reservable campus (e.g. `Intramuros Campus` , `Makati Campus`), seeded by osaar on the `/campus` page. The SAAF reservation step and the `activity_details.venue` dropdown read the list.

Property	Type	Notes
`PK`	string	`CAMPUS#uuid`
`SK`	string	`CAMPUS#uuid` (same as PK)
`name`	string	Display name
`classroom_name_prefixes`	list	**Optional.** Allowed classroom-name prefix tokens (1–10 letters each, up to 30; normalized uppercase + deduped). Stored only when paired with `classroom_name_digits`
`classroom_name_digits`	number	**Optional.** Required digit count (1–4). With the prefix list it defines how this campus's classroom rooms are named (e.g. `MPO` + 3 → `MPO123`)

List scans `PK` begins `CAMPUS#` AND `PK = SK` . Delete is guarded: **409** if any RESERVABLE still lives under `PK = CAMPUS#id` . Rename in place (PUT) to keep the same uuid so RESERVABLE children and stored booking `campus_id` snapshots stay valid. The classroom format is authored on the osaar `/campus` Add/Edit page; a campus without it cannot host classroom rooms.

RESERVABLE

A bookable room or equipment item under a campus. Its `SK` is globally unique and doubles as the BOOKING partition key.

Property	Type	Notes
`PK`	string	`CAMPUS#uuid` (parent campus)
`SK`	string	`RESERVABLE#uuid`
`name`	string	
`type`	string	`room` | `equipment`
`schedule`	object	Recurring weekly template: `monday` .. `saturday`, each exactly 12 booleans (one per 07:00–21:00 / 70-min slot; `true` = available). Sunday is never reservable; an absent day is all-unavailable
`min_participants`	number	**Room-only, optional.** Lower bound on the expected headcount a room can hold. Omitted/unset = no lower limit; never stored for `equipment`
`max_participants`	number	**Room-only, optional.** Upper bound on the expected headcount a room can hold. Omitted/unset = no upper limit; never stored for `equipment`
`is_classroom`	boolean	**Room-only, optional.** True marks the room as a classroom whose `name` must match the owning campus's classroom format. Stored only when `type` is `room` and the flag is true; read back as false otherwise

Added/edited by cdm on the `/reservables` page (CSV for name+type, a grid for the schedule; participant bounds are room-only inputs, disabled and dropped for equipment). The `is_classroom` toggle is room-only and shown-but-disabled until the selected campus defines a classroom format; when enabled the room `name` is validated against `<one prefix><exactly N digits>` at create/update (**422** on the name if it misses, or **422** on `is_classroom` if the campus has no format). On SAAF submit the activity's `expected_participants` is checked against each picked room's bounds (`BookingRecords::assertAvailable`); an out-of-range headcount aborts with **422** before any BOOKING is written. Delete is guarded: **409** if any BOOKING still occupies `PK = RESERVABLE#id` .

BOOKING

A concrete hold on a reservable's slots for specific dates. One BOOKING per (reservable, submission) or per (reservable, CDM reserve action). Two flavors share the item type, distinguished by `source` .

Property	Type	Notes
`PK`	string	`RESERVABLE#uuid`
`SK`	string	`BOOKING#uuid`
`timestamp`	timestamp	Create time (UTC)
`schedule_selected`	list	`[{ date: "YYYY-MM-DD", slots: [int 0-11] }]` (normalized)
`source`	string	`submission` (SAAF-created hold) | `cdm` (manual CDM hold)
`campus_id`	string	Bare campus uuid (snapshot)
`reservable_name`	string	Snapshot at write time
`reservable_type`	string	`room` | `equipment` (snapshot)

Submission-sourced (`source: "submission"`) adds GSI5 + traceability, and is released on deny/return/re-edit via the SUBMISSION's `booking_refs` :

Property	Type	Notes
`GSI5PK`	string	`ORGANIZATION#uuid` — owning org (org booking list)
`GSI5SK`	timestamp	`timestamp`
`event_id` / `submission_id` / `organization_id`	string	Bare uuids for traceability and read-only CDM labelling

Manual (`source: "cdm"`) carries **no** GSI5 and **no** submission linkage (so it never surfaces in an org's list); it is released only by CDM delete:

Property	Type	Notes
`booked_by`	string	`SIGNATORY#uuid` (the CDM signatory)
`reason`	string	Optional purpose note (omitted when blank)

Both flavors share one availability engine: a slot is free only when the weekly template allows it AND no BOOKING (either source) occupies that date+slot. Conflict detection runs on the base RESERVABLE partition — never GSI5 — so it works before the (operator-provisioned) organization index exists.

A submission-sourced hold is also **edited in place** through a RESERVATION_CHANGE resolution: a same-reservable date/slot change patches `schedule_selected` on the existing `PK` / `SK` (keys, GSI5 linkage and `submission_id` / `event_id` / `organization_id` all survive, and the SUBMISSION's `booking_refs` entry stays valid); a room swap deletes the old BOOKING and recreates a `source:"submission"` hold under the new RESERVABLE partition, swapping the `booking_refs` entry to the new keys in the same submission update.

RESERVATION_CHANGE

A student's request to cancel or reschedule the holds on an **approved** paper (which the student cannot edit directly), resolved by CDM. Each request freezes the concrete BOOKING records it targets and keeps a full before/after audit trail once resolved.

Property	Type	Notes
`PK`	string	`EVENT#uuid` (parent event partition)
`SK`	string	`RESERVATION_CHANGE#uuid`
`item_type`	string	`reservation_change`
`organization_id` / `submission_id` / `campus_id`	string	Bare uuids, snapshotted at request time
`requested_by`	string	`STUDENT#<sub>` from the JWT
`requested_at`	timestamp	
`request_type`	string	`cancel` | `reschedule`
`reason`	string	Required (≤500)
`suggested_date`	string	**Sparse, reschedule only.** `"YYYY-MM-DD"`, advisory — CDM is not bound by it
`bookings[]`	list	The targeted holds, **frozen** from the paper's live `booking_refs` at request time: `{ pk, sk, reservable_id, reservable_name, reservable_type, schedule_selected[] }`
`status`	string	`pending / resolved / denied`
`GSI6PK`	string	`CAMPUS#uuid` — CDM per-campus queue
`GSI6SK`	string	`status#requested_at` (`pending#...`, `resolved#...`, `denied#...`)

On resolution the item also carries `decision` (`cancel / reschedule / deny`), `resolved_by` (`SIGNATORY#uuid`), `resolved_at`, `comment`, and an `applied` audit object:

Property	Type	Notes
`applied.bookings[]`	list	One entry per targeted hold: `{ pk, sk, action: released | kept | updated | replaced, before: { reservable_id, schedule_selected }, after: {…} | null, new_pk, new_sk }` — `new_pk` / `new_sk` only on a room swap (point at the recreated BOOKING)
`applied.submission`	object | null	`{ status_before, status_after, date_of_event_before/after, time_of_event_before/after, booking_refs_before, booking_refs_after }`; `null` for a deny

`applied` is an audit trail only — never read back as the source of truth. One pending request per paper. The student side lists the full history with a base-table query on `PK = EVENT#uuid`, `SK` begins `RESERVATION_CHANGE#` (no GSI); the CDM queue reads **GSI6** (operator-provisioned in AWS, mirroring GSI5; the CDM list endpoint fails loudly until it exists). Resolve is guarded: the request must still be `pending` (`finalize` uses a conditional put on `attribute_not_exists(resolved_at)` so two CDM users cannot resolve it twice), the paper must still be `approved`, and every snapshotted hold must still appear in the paper's live `booking_refs` (a stale request aborts **409** with zero writes). The request window closes at the end of `activity_details.date_of_event` in Asia/Manila; CDM may resolve at any time.

mapua-apex — DynamoDB Access Patterns

DynamoDB is modeled around **access patterns**, not normalized relations: every query is designed first, then the partition key (PK), sort key (SK), and global secondary indexes (GSI) are chosen to serve it. All patterns hit a **single table**. There are no joins — related IDs are copied onto items at write time.

Index summary:

Index	Key	On	Purpose
Base table	`PK` / `SK`	all items	Primary access
GSI1	`GSI1PK` / `GSI1SK`	EVENT, SUBMISSION	Org-scoped event list & admin submission list
GSI2	`GSI2PK` / `GSI2SK`	SUBMISSION	Sparse signatory inbox (`pending` / `returned`)
GSI4	`GSI4PK` / `GSI4SK`	SIGNATORY	Role directory (`ROLE#...`)
GSI5	`GSI5PK` / `GSI5SK`	BOOKING	Org-scoped booking list (submission-sourced holds only; operator-provisioned)
GSI6	`GSI6PK` / `GSI6SK`	RESERVATION_CHANGE	CDM per-campus reservation-change queue (`status#requested_at` ; operator-provisioned)

Core access patterns

#	Access pattern	Operation	Key condition	Index
1	Org event list (student dashboard)	Query	`GSI1PK = ORGANIZATION#id` , `GSI1SK` desc (first 25 EVENT items)	GSI1
2	Event children (its submissions)	Query	`PK = EVENT#id` , `SK` begins `SUBMISSION#`	Base
3	Student's own submissions	Query	GSI1 for org, then each event partition	GSI1 + Base
4	Collaboration submissions (dependent view)	Query	`PK = ORGANIZATION#<dep>` , `SK` begins `COLLAB#` , then resolve each pointer to its master SUBMISSION	Base

#	Access pattern	Operation	Key condition	Index
5	Single submission detail	GetItem	`PK = EVENT#id`, `SK = SUBMISSION#id`	Base
6	Submission timeline / audit	Query	`PK = SUBMISSION#id`, `SK` begins `NOTIFICATION#`	Base
7	Signatory inbox (desk queue)	Query	`GSI2PK = SIGNATORY#id`, `GSI2SK` by time (sparse)	GSI2
8	Signatory directory by role	Query	`GSI4PK = ROLE#...`, `GSI4SK` begins `SIGNATORY#`	GSI4
9	Current signatory profile (`/signatories/me`)	GetItem	`PK = SIGNATORY#id`, `SK = SIGNATORY#id`	Base
10	Admin: all submissions	Query	per org `GSI1PK = ORGANIZATION#id`, `GSI1SK` begins `SUBMISSION#`	GSI1
11	Student org record (`/students/organization`)	GetItem	`PK = ORGANIZATION#id`, `SK = ORGANIZATION#id`	Base
12	Org directory (`/students/organizations`)	Query	org list → `{ organization_id, name }`	Base
13	Announcement list	Query	`PK = ANNOUNCEMENT`, `SK` desc (newest first)	Base
14	Announcement show / edit / delete	GetItem / PutItem / DeleteItem	`PK = ANNOUNCEMENT`, `SK = timestamp`	Base
15	Write submission (+ collab pointers)	PutItem	`PK = EVENT#id`, `SK = SUBMISSION#id`; maintain one COLLAB pointer per dependent	Base

#	Access pattern	Operation	Key condition	Index
16	Approve / return / deny	UpdateItem	advance `current_signatory` / `GSI2PK` from stored `signatory_sequence`; write NOTIFICATION	Base
17	Admin: delete organization (guarded)	Query → DeleteItem	block (**409**) if `GSI1PK = ORGANIZATION#id` has events/submissions or base `PK = ORGANIZATION#id`, `SK` begins `COLLAB#`; else DeleteItem `PK = SK = ORGANIZATION#id`	GSI1 + Base
18	Admin: delete signatory (guarded)	Scan/Query → DeleteItem	block (**409**) if any `ORGANIZATION.signatories` desk holds the id or `GSI2PK = SIGNATORY#id` (open inbox); else DeleteItem `PK = SK = SIGNATORY#id`	GSI2 + Base
19	Campus list (osaar `/campus`, SAAF venue dropdown)	Scan	`PK` begins `CAMPUS#` AND `PK = SK`	Base
20	Reservables under a campus	Query	`PK = CAMPUS#id`, `SK` begins `RESERVABLE#`	Base
21	Reservable availability (template + per-date free/booked slots)	Query	`PK = RESERVABLE#id`, `SK` begins `BOOKING#`; overlay bookings on the weekly template across the window	Base
22	CDM View/Reserve calendar (raw bookings, both sources)	Query	`PK = RESERVABLE#id`, `SK` begins `BOOKING#` (filtered to dates in window)	Base

#	Access pattern	Operation	Key condition	Index
23	Create booking (submission hold or CDM manual)	Query → PutItem	validate selections vs template + existing bookings (**409** on conflict) and, on submission, expected headcount vs each room's `min/max_participants` (**422** out of range), then PutItem `PK = RESERVABLE#id`, `SK = BOOKING#id`	Base
24	Release holds on deny / return / re-edit / cancel-resolution	DeleteItem	delete each `booking_refs[] {pk, sk}` (submission holds)	Base
25	Delete a manual booking (CDM)	GetItem → DeleteItem	**409** if `source` is `submission`; else DeleteItem `PK = RESERVABLE#id`, `SK = BOOKING#id`	Base
26	Admin: delete campus / reservable (guarded)	Query → DeleteItem	block (**409**) if a campus owns any RESERVABLE, or a reservable owns any BOOKING; else DeleteItem	Base

#	Access pattern	Operation	Key condition	Index
27	Create / edit reservable (CDM Add/Edit)	GetItem → PutItem	for a `room` flagged `is_classroom`, GetItem the owning CAMPUS and enforce it defines a classroom format (**422** on `is_classroom` if not) and the `name` matches `<prefix><N digits>` (**422** on `name`), then PutItem `PK = CAMPUS#id`, `SK = RESERVABLE#id`; room-only `min/max_participants` + `is_classroom` dropped for `equipment`	Base
28	Student: request reservation change	Query → PutItem	guard `status = approved`, live `booking_refs`, within the event-day window (Asia/Manila) and no pending request (base query `PK = EVENT#id`, `SK` begins `RESERVATION_CHANGE#`) — **409** otherwise; then PutItem `PK = EVENT#id`, `SK = RESERVATION_CHANGE#id` freezing the `bookings[]` snapshot + write NOTIFICATION	Base
29	Student: list reservation changes	Query	`PK = EVENT#id`, `SK` begins `RESERVATION_CHANGE#` (newest first)	Base
30	CDM: reservation-change queue	Query	`GSI6PK = CAMPUS#id`, optional `status` filter, `GSI6SK` newest-first	GSI6

#	Access pattern	Operation	Key condition	Index
31	CDM: resolve reservation change	Query → Put/Update/DeleteItem	guard request still `pending` (conditional put `attribute_not_exists(resolved_at)`), paper still `approved`, and snapshot ⊆ live `booking_refs` (**409** stale). **cancel**: delete every hold, patch SUBMISSION `status = cancelled` + drop `booking_refs`. **reschedule**: validate new selections (**409** conflict / **422** bounds), patch (`update`) or delete+recreate (`swap`) each BOOKING, rewrite `venue_reservation` / `booking_refs` / event timing in one submission update. **deny**: paper untouched. All write the `applied` audit + NOTIFICATION	Base

---

Page / feature → objects accessed

Page / feature	Objects read or written	Patterns
Student dashboard	ORGANIZATION, EVENT, SUBMISSION, ANNOUNCEMENT, COLLAB POINTER	1, 2, 3, 4, 11, 13
Submission tracker / detail	SUBMISSION, NOTIFICATION, SIGNATORY (by copied id)	5, 6
New / edit submission (SAAF)	SUBMISSION, COLLAB POINTER, ORGANIZATION (directory), CAMPUS, RESERVABLE, BOOKING (holds + `booking_refs`)	12, 15, 19, 20, 21, 23, 24
Signatory desk (inbox)	SUBMISSION, SIGNATORY	7, 9
Signatory review (approve/return/deny/classify)	SUBMISSION, NOTIFICATION, BOOKING (release holds on deny/return)	6, 16, 24

Page / feature	Objects read or written	Patterns
Admin dashboard (submissions)	ORGANIZATION, SUBMISSION	10
Admin → Organizations	ORGANIZATION, SIGNATORY	11, 12, 17
Admin → Signatories	SIGNATORY, ORGANIZATION (desk guard)	8, 18
osaar → Campus	CAMPUS, RESERVABLE (delete guard)	19, 20, 26
cdm → Reservables (Add/View/Reserve)	CAMPUS, RESERVABLE, BOOKING	20, 21, 22, 23, 25, 26, 27
Student dashboard → request change (dialog)	SUBMISSION, RESERVATION_CHANGE, BOOKING (snapshot), NOTIFICATION	6, 28, 29
cdm → Reservations (resolve cancel / reschedule / deny)	RESERVATION_CHANGE, SUBMISSION, BOOKING, RESERVABLE, CAMPUS, NOTIFICATION	20, 21, 23, 24, 30, 31
About / Announcements	ANNOUNCEMENT	13, 14